

Peregrine Framework API Reference

This is the exhaustive API Reference for the Peregrine C++ Web Framework. It documents every core module, public class, available method, and provides detailed code examples for all functionalities.

1. `app.hpp` — Application Core

The `peregrine::App` (aliased as `peregrine::Peregrine`) is the central registry for your application. It handles routing, middleware, and runs the multi-threaded HTTP server.

Class: `Peregrine`

Constructors

- `Peregrine(std::string name, std::string static_folder = "", std::string static_url = "")`
 - **name**: The name of the application.
 - **static_folder**: Directory to serve static files from (optional).
 - **static_url**: URL prefix for static files (optional).

Configuration Properties

- `Config config`: A dictionary-like object to store app-wide configuration.
- `std::string secret_key`: The cryptographic key used to sign secure session cookies. **Must be set to use sessions.**
- `std::vector<std::string> secret_key_fallbacks`: Keys used for rotating secrets without invalidating existing sessions.
- `std::string session_cookie_name`: Name of the session cookie (default: `"session"`).
- `int session_lifetime_seconds`: Lifespan of a permanent session (default: `86400`).

Security Methods

- `void enable_csrf_protection(std::vector<std::string> exempt_paths = {})`
 - Enables Cross-Site Request Forgery protection. When enabled, any mutating method (`POST`, `PUT`, `DELETE`) will check for a valid CSRF token unless the path is in the `exempt_paths` list.

Routing Methods

All routing methods take a path pattern and a handler lambda of signature `[] (Request&`

```
req) -> Response .
```

- `void get(const std::string& path, Handler handler)`
- `void post(const std::string& path, Handler handler)`
- `void put(const std::string& path, Handler handler)`
- `void patch(const std::string& path, Handler handler)`
- `void del(const std::string& path, Handler handler)`
- `void options(const std::string& path, Handler handler)`
- `void route(const std::string& path, const std::vector<std::string>& methods, Handler handler)`

◦ Allows registering a single handler to multiple HTTP methods.

Example:

```
app.route("/api/upload", {"POST", "PUT"}, [](Request& req) {  
    return Response::text("File Uploaded");  
});
```

Blueprint Registration

- `void register_blueprint(const Blueprint& bp, const std::string& url_prefix_override = "")`

◦ Attaches a blueprint module to the main application.

Static File & SPA Serving

- `void static_dir(const std::string& url_prefix, const std::string& directory_path)`
 - Maps a URL prefix (e.g., `/assets`) to a physical directory (e.g., `./public/assets`).
- `void serve_spa(const std::string& dist_dir = "dist", const std::string& index_file = "index.html", const std::string& api_prefix = "api")`
 - Intelligently serves a Single Page Application (React/Vue). Routes unknown paths to `index.html` unless the path starts with `api_prefix`, in which case it returns a 404 JSON error.

Server Execution

- `void run(const std::string& host, int port)`
 - Starts the HTTP server on the given IP address and port.
- `void run(const ServerConfig& config)`
 - Starts the server using advanced configurations (like HTTPS/TLS support).

2. `request.hpp` — Request Handling

The `peregrine::Request` object encapsulates all incoming data from the client. It is passed by reference (`Request& req`) to every route handler.

Class: Request

Standard Properties

- `std::string method` : The HTTP verb (`"GET"` , `"POST"` , etc.).
- `std::string path` : The URL-decoded path (e.g., `"/users/1"`).
- `std::map<std::string, std::string> path_params` : Extracted variables from the URL router (e.g., `req.path_params["id"]`).
- `std::string body` : The raw string of the HTTP request body.
- `Session session` : The secure cookie session object (see `session.hpp`).
- `std::map<std::string, std::string> g` : A generic map to store context data for the duration of this specific request (useful in `before_request` middleware).

Data Retrieval Methods

- `std::string header(const std::string& name, const std::string& def = "") const`
 - Gets an HTTP header (case-insensitive).
- `std::string arg(const std::string& name, const std::string& def = "") const`
 - Gets a variable from the URL query string (e.g., `?sort=desc`).
- `std::string form_get(const std::string& name, const std::string& def = "") const`
 - Gets a field from a submitted `application/x-www-form-urlencoded` or multipart form.
- `std::string cookie(const std::string& name, const std::string& def = "") const`
 - Reads a raw cookie sent by the client.
- `Json get_json() const`
 - Parses the `body` string into a `Json` object. Returns an empty object if parsing fails.

File Uploads (`types.hpp`)

- `std::vector<UploadedFile> files` : A vector containing all files uploaded via `multipart/form-data` .
 - `UploadedFile` struct properties: `field_name` , `filename` , `content_type` , `data` .
 - `bool UploadedFile::save(const std::string& path) const` : Writes the file to disk.

Example: Processing a Request

```
app.post("/submit", [](Request& req) {
    if (req.is_json()) {
        auto data = req.get_json();
        return Response::text("JSON Name: " + data["name"].string_value());
    } else {
```

```

        return Response::text("Form Name: " + req.form_get("name"));
    }
});

```

3. `response.hpp` — Response Generation

Handlers must return a `peregrine::Response` object. The class provides convenient static factories.

Class: `Response`

Properties

- `int status` : The HTTP status code (default `200`).
- `std::map<std::string, std::string> headers` : Outgoing HTTP headers.
- `std::string body` : The raw response body.

Static Factories

- `static Response text(const std::string& body_text, int status_code = 200)`
- `static Response html(const std::string& html_content, int status_code = 200)`
- `static Response json(const Json& j, int status_code = 200)`
- `static Response redirect(const std::string& location, int status_code = 302)`
- `static Response stream(const std::string& content_type, std::function<void(Connection&)> streamer_fn, int status_code = 200)`
 - Initiates an HTTP chunked or continuous data stream.
- `static Response mjpeg(std::function<void(Connection&)> streamer_fn)`
 - Specialized stream for sending live MJPEG video feeds (e.g., for IP Cameras).

Cookie Management

- `Response& set_cookie(name, value, max_age_seconds=-1, path="/", domain="", http_only=false, secure=false, same_site="", partitioned=false)`
 - Appends a `Set-Cookie` header.
- `void delete_cookie(name, path="/", domain="")`
 - Clears a cookie by setting its Max-Age to 0.

Mutators (Chainable)

- `Response& set_header(const std::string& key, const std::string& value)`
- `Response& set_status(int status_code)`

Example: Building a Response

```
app.get("/api/user", [](Request& req) {
    auto res = Response::json({{"name", "Alice"}});
    return res.set_status(201).set_header("X-RateLimit", "50");
});
```

4. `blueprint.hpp` — Modularity

Blueprints allow you to split your application into multiple files.

Class: `Blueprint`

Constructor

- `Blueprint(std::string name, std::string url_prefix = "")`

Routing Methods

Blueprints possess the exact same routing methods as the `App` class (`get` , `post` , `put` , `del` , `route`).

Middleware Hooks (Blueprint Local)

- `void before_request(std::function<void(Request&)> fn)`
 - Runs *only* for routes registered on this blueprint.
- `void after_request(std::function<void(Request&, Response&)> fn)`
 - Runs *only* for routes registered on this blueprint.

Example: Creating and Registering a Blueprint

```
Blueprint auth_bp("auth", "/auth");

auth_bp.post("/login", [](Request& req) {
    return Response::text("Logged In");
});

// Inside main.cpp
app.register_blueprint(auth_bp); // Route is now /auth/login
```

5. `session.hpp` — Secure State Management

Peregrine implements cryptographically signed cookies to store state securely on the client

Peregrine implements cryptographically signed cookies to store state securely on the client.

Class: `Session`

You do not instantiate this yourself. You access it via `req.session`.

Methods

- `std::map<std::string, std::string>& data()`
 - Returns the mutable map containing session variables. Reading or modifying this map automatically marks the session as accessed or modified.
- `std::string get(const std::string& key, const std::string& default_val = "")`
 - Safely fetches a value.
- `void clear()`
 - Deletes all session data.

Properties

- `bool permanent`
 - If set to `true`, the session cookie will persist for `app.session_lifetime_seconds` (default 24h). If `false`, the cookie deletes when the user closes their browser.

Example: Session Management

```
app.post("/login", [](Request& req) {
    req.session.data()["user_id"] = "999";
    req.session.permanent = true; // Remember me across browser restarts
    return Response::redirect("/dashboard");
});

app.post("/logout", [](Request& req) {
    req.session.clear();
    return Response::redirect("/");
});
```

6. `template.hpp` — Render Engine

Peregrine includes a fast, lightweight template engine for HTML generation.

Class: `TemplateContext`

A data container passed to the template engine.

- `TemplateContext& set(const std::string& key, const std::string& value)`
 - Defines a variable accessible via `{{key}}` in the template.

- `TemplateContext& append(const std::string& list_name, const std::map<std::string, std::string>& item)`
 - Adds an item to a list accessible via the `{{#each list_name}} ... {{this.field}} ... {{/each}}` block in the template.

Rendering Functions

- `std::string render_string(const std::string& tpl_string, const TemplateContext& ctx)`
- `std::string render_template(const std::string& file_path, const TemplateContext& ctx)`

Example: Rendering HTML

```
// In index.html:
// <h1>Hello {{name}}</h1>
// <ul>
// {{#each users}} <li>{{this.username}}</li> {{/each}}
// </ul>

app.get("/", [] (Request& req) {
    TemplateContext ctx;
    ctx.set("name", "Admin");
    ctx.append("users", {"username", "Alice"});
    ctx.append("users", {"username", "Bob"});

    std::string html = render_template("templates/index.html", ctx);
    return Response::html(html);
});
```

7. `types.hpp` & Configuration

Protocol Configuration

When starting the server via `app.run(ServerConfig config)`, you can configure HTTPS and max limits.

```
ServerConfig config;
config.protocol = Protocol::HTTPS;
config.host = "0.0.0.0";
config.port = 443;
config.cert_file = "/path/to/cert.pem";
config.key_file = "/path/to/key.pem";
config.max_body_size = 50 * 1024 * 1024; // 50 MB upload limit

app.run(config);
```

JSON Handling

Peregrine uses a robust, built-in JSON library (`peregrine::Json`).

- Creation: `Json j = Json::object {{"name", "Test"}, {"id", 1}};`
 - Array: `Json a = Json::array {1, 2, 3};`
 - Serialization: `j.dump()`
 - Deserialization: `Json::parse(string)`
-

End of API Reference